DAY 29 / DATABRICKS LEARNING SERIES

END-TO-END DATABRICKS ARCHITECTURE

A Practical Study Guide with an E-Commerce Case Study

From source systems to ingestion, Bronze, Silver, Gold, BI, ML and production operations.

What you will learn

- How data physically moves from source systems into governed Delta tables
- How to design Bronze, Silver, and Gold with the correct grain and history
- Where Auto Loader, Lakeflow Connect, Lakeflow pipelines, Jobs and Unity Catalog fit
- How analytics, ML, CI/CD, data quality, security and recovery connect
- How to defend your design choices in a real architecture interview

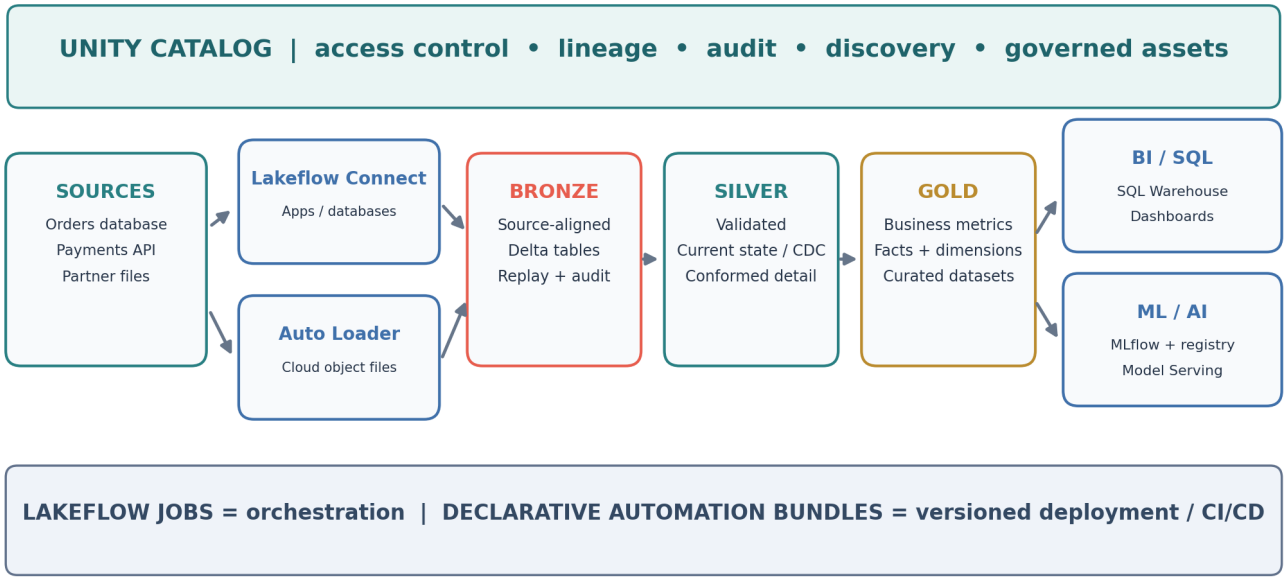
TECHNICAL LEVEL Beginner – Intermediate – Interview-ready

VERIFIED AGAINST OFFICIAL DATABRICKS DOCUMENTATION 10 OCTOBER 2026

SCOPE These are topic notes for learning and reference—not an on-camera script. Examples are illustrative and require environment-specific access, credentials and deployment configuration.

01 The Big Picture

An end-to-end data architecture is more than a sequence of tools. It is a system for collecting data, preserving its meaning, improving quality, serving business questions, and operating reliably under security and cost constraints.



Compute processes the data; Delta tables live on cloud storage such as S3 / ADLS / GCS. Diagram is conceptual, not a mandated layout.

Figure 1. An example e-commerce architecture: sources → ingestion → Delta medallion layers → BI/ML, with governance and operations across the lifecycle.

Five building blocks

Building block	Question it answers	Typical Databricks capability
Source + ingest	Where does data originate? How is it captured?	Lakeflow Connect, Auto Loader, streaming / APIs
Store + transform	How is raw data retained, validated and modeled?	Delta Lake, Lakeflow pipelines / Spark
Serve	How do people or applications consume it?	SQL Warehouses, dashboards, ML / Model Serving
Govern	Who can discover, access and trace it?	Unity Catalog
Operate	How does it run, deploy, alert and recover?	Lakeflow Jobs, Declarative Automation Bundles

IMPORTANT DISTINCTION "Bronze/Silver/Gold" describe data-quality and consumption layers. They are not Databricks compute products. A Unity Catalog catalog/schema can be organized by environment, layer, source, or business domain; the medallion pattern is recommended but optional.

Where the bytes actually live

For a cloud lakehouse, Delta table data and transaction logs live in cloud object storage (such as S3, ADLS Gen2, or GCS). Spark/SQL compute reads and writes that storage. Unity Catalog governs registered assets, privileges and metadata; it is not a substitute for storage. Databricks also has a control-plane / compute-plane architecture, which is different from the Bronze/Silver/Gold data pattern.

02 Business Case: E-Commerce Company

Imagine an online retailer with four sources and two outcomes. Leadership needs daily net revenue and fulfillment dashboards. The data-science team wants a fraud-risk model. The architecture must keep change history, handle duplicate deliveries, protect customer data, and be rerunnable.

System	Example input	Change pattern	Latency goal
Orders SQL database	orders, order_items, customers	Inserts / updates / deletes (CDC)	15–30 min
Payment platform API	authorizations, captures, refunds	Append/event updates + pagination	15–30 min
Partner object storage	inventory_YYYYMMDD.csv	New CSV files each day	Daily
App events (optional)	page views, add-to-cart events	Kafka / event stream	Near-real-time

Illustrative source records

```
orders:    order_id=O-101, customer_id=C-7, status=SHIPPED, amount=1200.00, updated_at=10:05
payments:  payment_id=P-8, order_id=O-101, type=CAPTURE, amount=1200.00
inventory: SKU-20, partner=SUP-2, available_qty=34, file_date=2026-10-10
```

Key architecture questions before tool choice: What is the primary business event? What is each table's grain? Is the source a full extract, append log or CDC feed? What must be retained for audit? What is the SLA? What personal data must be restricted? What happens if a job reruns?

03 Ingestion: Getting Data into the Lakehouse

A. Database / business application → Lakeflow Connect

Use a supported managed Lakeflow Connect connector when its source and release status match your requirements. For orders SQL Server (where supported), a connector can manage initial loading and subsequent incremental changes. The managed connector produces ingestion pipelines using Lakeflow pipelines and Unity Catalog governance. Connector behavior and supported operations vary by source and release status.

1. Capture initial data or a consistent starting state of the source table.
2. Read subsequent incremental changes (where the connector offers CDC / incremental capture).
3. Write the ingested results to an appropriately governed target table.

4. Track pipeline state, connector health, update lag and source permissions.

B. Partner files → Cloud storage → Auto Loader

Auto Loader is a file-incremental ingestion mechanism available through the cloudFiles source. It identifies newly arrived files and stores processing progress in a checkpoint (or pipeline-managed state). It avoids repeatedly processing previously completed files under normal, correctly configured operations; do not describe it as a generic scanner that "never reprocesses anything."

```
from pyspark import pipelines as dp

@dp.table(name="bronze_partner_inventory")
def bronze_partner_inventory():
    return (spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", "true")
            .load("s3://example-landing/partner/inventory/"))
```

Illustrative Lakeflow pipeline definition. Configure permissions, schema management, target catalog/schema and file event strategy for your workspace. For a standalone Structured Streaming job, configure a durable checkpointLocation and, if inferring/evolving schemas, a schemaLocation. Lakeflow pipelines manages checkpoint and schema state in the pipeline context.

Understand three independent concerns

Concern	Why it matters	Design response
Discovery progress	Which files have been seen / processed?	Stable checkpoints; recover rather than reset casually
Schema drift	CSV adds discount_code tomorrow	Explicit schema / schema evolution / rescued data policy
File mutation	A supplier overwrites yesterday's file	Define immutability and reprocessing contract; understand allowOverwrites

CHECKPOINT CAUTION Changing or deleting a streaming checkpoint can cause a new stream to treat old input as unseen. File-level processing guarantees do not automatically equal business-record deduplication—duplicates may exist across separate files.

04 Bronze: Preserve Evidence and Replayability

Bronze is your high-fidelity, source-aligned ingestion layer. Its primary purpose is to retain what arrived with sufficient metadata to diagnose, backfill and reprocess into a better downstream implementation. Keep business decisions out of Bronze wherever possible.

Representative Bronze tables

Table	Representative columns	Storage / behavior
bronze_orders_cdc	order_id, status, amount, op, source_lsn, source_ts, _ingested_at	Delta append log of source changes
bronze_payments_events	payment_id, order_id, event_type, amount, occurred_at, payload	Delta events from API / landing
bronze_inventory_files	sku, partner_id, available_qty, _source_file, _ingested_at	Delta rows from immutable partner files

Technical metadata is useful: _ingested_at, _source_file, _batch_id, _source_system, source change sequence, and load timestamp. Use source-appropriate fields; file paths are irrelevant for an API without files.

What Delta Lake contributes

- Parquet data files + a Delta transaction log (_delta_log): readers identify a consistent committed snapshot, not "the latest Parquet by timestamp."
- ACID transactions, schema controls and time travel within retention policy.
- MERGE, UPDATE and DELETE on Delta tables, and optional Change Data Feed for downstream changes.
- A simple audit/replay foundation—but replayability is only as complete as your retained source history and retention policy.

NUANCE: "BRONZE = APPEND-ONLY" Append-oriented Bronze is a strong default for raw events and CDC logs, but it is not a universal platform rule. Managed connectors and snapshot ingestion patterns may maintain a current-state target. Explicitly decide whether you need immutable raw history, current state, or both; never assume a changing snapshot is a complete audit log.

05 Silver: Trusted, Detailed Business Entities

Silver applies validation and business-safe standardization while retaining useful detail. It is where you turn messy source records into reliable entities: orders, order items, payments, products, customers and inventory.

Typical Silver rules for the example

- Type and normalize amount, currency, timestamps and identifiers.
- Deduplicate by the real source business key plus event/version sequence (not arbitrary dropDuplicates on every column).
- Apply CDC updates/deletes in source order; explicitly define how late and out-of-order changes behave.
- Quarantine invalid records or fail critical updates instead of silently dropping revenue events.

- Normalize status vocabularies and join keys; keep orders and payments at their own natural grains.

CDC decisions: current state versus history

Pattern	Silver target	Use it when
SCD Type 1	Latest known values per key	Current order/customer attributes matter
SCD Type 2	Versions with effective boundaries	Need to analyze historical customer attributes
Event / ledger model	One record per business event	Payment capture/refund or audit history

Lakeflow pipelines provide declarative transformations and Databricks-specific AUTO CDC capabilities for SCD1 and SCD2. This differs from plain Apache Spark Declarative Pipelines (SDP), which defines the portable base but not all Databricks features.

Practical expectations example

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col

@dp.materialized_view(name="silver_valid_payments")
@dp.expect_or_drop("has_payment_id", "payment_id IS NOT NULL")
@dp.expect_or_fail("valid_amount", "amount >= 0")
def silver_valid_payments():
    return (spark.read.table("commerce_dev.bronze.payments_events")
            .select("payment_id", "order_id", "amount", "event_type"))
```

Demonstration of expectation behavior, not a complete payment-ledger implementation. Positive amounts can still be associated with refunds when event_type carries the sign. Choose rules based on source semantics.

EXPECTATION MODES WARN retains invalid rows and records metrics; DROP excludes violating rows; FAIL stops the affected pipeline update. For finance-critical records, prefer quarantine and reconciliation rather than making lost rows invisible.

06 Gold: Design from Business Questions and Grain

Gold is the business-facing consumption layer. Do not start by asking whether a table “needs a fact and dimension.” Start with a business question, define the event and grain, list required measures, then decide the right model: curated detailed table, dimensional model, wide serving table, or aggregate.

A clean analytics model for the retailer

Dataset	Grain	Purpose

gold.fact_order_items	One row per order line item	Sales by SKU, category, date, location
gold.fact_payments	One row per payment event	Captured payments, refunds, payment status
gold.dim_product	One row per product or product version	Category and product attributes
gold.dim_customer	One row per customer / effective version	Segment / cohort / lifecycle
gold.daily_net_sales	One row per date + channel	Fast executive dashboard



Example only: a gold star schema is useful for BI; it is not mandatory for every gold dataset.

Figure 2. Example dimensional choice for a sales BI use case; the product and date dimensions do not erase the base event grain.

Avoid metrics that look right but are wrong

- Joining each order to multiple payment events and multiple order lines can multiply rows (fan-out). Aggregate each fact to a compatible grain before combining.
- "Revenue" requires a definition: ordered, booked, shipped, captured, refunded, canceled, taxes and FX are not interchangeable.
- One order may have many items, payments and refunds. Do not force these into a single wide fact without a documented grain.
- Gold does not have to be star schema for every workload. A compliance dashboard, search feature or ML dataset may need a curated flat/table-shaped output.

Illustrative daily net payments query

```
SELECT date(occurred_at) AS sales_date,
       SUM(CASE WHEN event_type = 'CAPTURE' THEN amount
                WHEN event_type = 'REFUND'  THEN -amount
                ELSE 0 END) AS net_captured_amount
FROM commerce_prod.silver.payment_events
WHERE event_type IN ('CAPTURE', 'REFUND')
GROUP BY date(occurred_at);
```

This is a simplified net cash-capture metric, not an accounting-standard net-revenue definition. Use currencies, local business cutoffs, chargebacks, reversals and reconciliation rules appropriate to the company.

07 Serving: Dashboards and AI Are Different Paths

Analytics path

Gold tables or materialized views are read by a Databricks SQL Warehouse, which supplies query compute for SQL consumers and BI dashboards. A warehouse does not replace the data tables; it executes and manages queries against them. Apply a SQL access role, sensible warehouse sizing, query optimization, and dashboard freshness targets.

Machine learning path

Data scientists may train from trusted Silver details, Gold features, or dedicated feature tables. MLflow tracks experiments, parameters, metrics and model artifacts. The Unity Catalog Model Registry manages registered model versions and governance. Databricks Model Serving (or another deployment option) exposes trained models for online inference. MLflow by itself is not a serving endpoint.

Step	ML example	Relevant capability
Prepare	Build order / customer fraud features	Silver / Gold / feature tables
Experiment	Train models; compare precision/recall	MLflow Tracking
Register	Promote a chosen model version	Unity Catalog Model Registry
Deploy	Expose real-time risk scoring	Databricks Model Serving
Operate	Watch drift, quality, latency and costs	Monitoring, Jobs, logs / metrics

DESIGN TAKEAWAY Do not make the model training lifecycle part of the daily BI refresh unless there is a genuine dependency. The same governed tables may support multiple independent consumers.

08 Governance: Unity Catalog as the Control Room

Unity Catalog is the governance plane for registered data and AI assets. It covers access control, data discovery, lineage and auditing, with security controls at the appropriate layers. The catalog hierarchy is typically catalog.schema.object.

```
commerce_prod.bronze.orders_cdc
commerce_prod.silver.orders_current
commerce_prod.gold.fact_order_items
commerce_prod.gold.daily_net_sales
```

Access policy for the example

Persona	Access posture	Why

Ingestion service principal	Write scoped ingest targets + required source access	Least-privilege automation
Data engineering group	Manage designated Bronze / Silver pipelines	Build and repair data products
BI analysts	SELECT in approved Gold datasets	Protect raw PII and unstable data
Data scientists	Approved Silver / Gold + model creation	Train governed models

For SELECT to work on a table, the principal generally also needs access to the parent catalog and schema (for example USE CATALOG and USE SCHEMA) according to the applicable privilege model.

```
GRANT USE CATALOG ON CATALOG commerce_prod TO `bi_analysts`;
GRANT USE SCHEMA ON SCHEMA commerce_prod.gold TO `bi_analysts`;
GRANT SELECT ON TABLE commerce_prod.gold.daily_net_sales TO `bi_analysts`;
```

Lineage and audit are not the same

- Lineage: orders_cdc → orders_current → fact_order_items → revenue_dashboard. It helps impact and root-cause analysis.
- Audit: who queried, modified, granted or performed an auditable action, subject to available system logs and retention.
- Fine-grained access: table privileges, row filters, masks, ABAC policies and workspace restrictions where applicable.
- Storage security still matters: cloud IAM, encryption, credentials, networking and external locations must also be configured.

GOVERNANCE CAVEAT Unity Catalog lineage is not a magic universal tracker for any arbitrary off-platform process. Automatic lineage depends on supported Databricks workloads, assets and permissions; add external metadata/lineage explicitly where needed.

09 Orchestration: Lakeflow Jobs

A pipeline is code and data dependencies. A job is an operational workflow that runs tasks on chosen compute, with schedules/triggers, dependencies, retries and notifications. Lakeflow pipelines can orchestrate transformations inside their declarative DAG; Lakeflow Jobs can orchestrate multiple pipelines, notebooks, SQL tasks and other workloads across the end-to-end system.

Suggested e-commerce production DAG

```
01_ingest_orders  -----+
01_ingest_payments -----+--> 02_silver_curate --> 03_data_quality
01_ingest_partner  -----+                              |
                                                       v
                                                04_publish_gold
                                                       |
                                              05_dashboard_freshness_check
```

Independent ingestion tasks can run in parallel. Make gold publication depend on reconciled Silver and successful critical checks. Use conditional tasks/branching where needed, and ensure repeated runs are idempotent.

Triggers, reliability and operations

- Run by schedule, file-arrival, table update, model update, continuous trigger, or manual/API invocation as supported.
- Use retries for transient failures, but make writes safe on reruns (stable keys, checkpoints, version-based merges).
- Capture run ID, input/output row counts, freshness, rejected records, duration and the owning team.
- Have an explicit recovery path: restart after transient failures, backfill date ranges, restore/rebuild from Bronze where retained.

10 CI/CD: From Code to Production Safely

The current Databricks product name is Declarative Automation Bundles (formerly Databricks Asset Bundles / DABs, renamed March 2026). Bundles declare project files and resources—jobs, pipelines and environment-specific settings—in source-controlled configuration. They enable validation, review and consistent deployment; the bundle is not an orchestration engine on its own.

Example project structure

```
ecommerce-lakehouse/
  databricks.yml
  resources/
    jobs.yml
    pipelines.yml
  src/
    bronze/
    silver/
    gold/
  tests/
    test_business_rules.py
  README.md
```

Minimal illustrative bundle manifest

```
bundle:
  name: ecommerce_lakehouse

include:
  - resources/*.yml

targets:
  dev:
    default: true
    mode: development
  prod:
    mode: production
```

Environment-specific workspace settings, identities, permissions and resources belong in the actual project configuration. The example intentionally omits sensitive URLs, credentials and job/pipeline resource definitions.

Promotion workflow

5. Feature branch → code review and unit tests (business transformations, data contracts).
6. Validate the bundle and the pipeline graph (where applicable).
7. Deploy to DEV and run integration / data-quality / reconciliation checks.
8. Promote reviewed configuration and code to PROD using an approved deployment identity.
9. Verify production resources, schedules, permissions and end-to-end data outputs.

```
databricks bundle validate -t dev
databricks bundle deploy   -t dev
# After gated release approval:
databricks bundle validate -t prod
databricks bundle deploy   -t prod
```

OPERATIONAL INSIGHT Deploying only notebooks while manually copying job YAML into production invites configuration drift. Managing jobs and pipelines as deployable resources makes changes auditable and reproducible.

11 Reliability, Performance, Security and Cost

Dimension	Production-grade decision	Failure to avoid
Freshness	SLA by source; lag alerts; backfill window	“Job succeeded” but data is stale
Data quality	Null / duplicate / reconciliation checks; quarantine	Silently drop bad payments
Idempotency	Checkpoints, source sequence, keys, MERGE rules	Double-count on retries
CDC correctness	Handle deletes, out-of-order and schema drift	Stale Silver customer/order state
Performance	Right compute; files, stats, clustering when justified	Unnecessary small files or scans
Cost	Triggered vs continuous; managed/serverless where fit	Idle compute / constant polling
Security	UC grants + IAM/network/secrets	Broad PII access
Observability	Run logs, metrics, lineage, ownership	No way to explain a revenue change

Where does optimization belong?

Optimize based on access pattern, not layer name. Bronze often favors ingestion throughput and retention; Silver may need clustering or rewrite strategies for expensive CDC joins; Gold often needs efficient selective queries and aggregates. Liquid clustering / predictive optimization applicability depends on table type, runtime and managed-table features. Validate performance with real query profiles before adding physical layout controls.

Two useful end-to-end checks

- Completeness: source count/change watermark → Bronze events → Silver valid/quarantined → Gold published counts. Document intentional exclusions.
- Business reconciliation: payment captures − refunds in the source ledger should reconcile with the equivalent Silver measure, within allowed lateness and currency rules.

12 Interview Walkthrough: Answer Like an Architect

"We have operational orders in SQL, payment events from an API, and inventory files from partners. I would start by clarifying SLAs, CDC availability, source grain, audit/PII constraints and target business metrics. Then I would choose source-appropriate ingestion, preserve source fidelity in Bronze, maintain validated entities in Silver, model consumer-specific Gold datasets, expose SQL/BI and ML consumers, apply Unity Catalog governance, and run the workload with monitored jobs and version-controlled deployments."

Questions the interviewer may ask

Question	Strong answer / trade-off
Why not write directly into Silver?	Source drift and quality errors can lose evidence; Bronze lets you replay and audit.
Auto Loader vs Connect?	Files in object storage vs managed/source-specific apps and databases.
Is Bronze always append-only?	Typically append-oriented; managed current-state snapshots also exist. Define retained history.
How do you process changed orders?	CDC with ordered changes, reliable keys, deletes and documented SCD semantics.
What if the same file arrives twice?	Checkpoint/file tracking plus independent record-key dedup and write idempotency.
Does every Gold table need facts/dims?	No. Use grain and consumer needs: BI star schema, aggregate, wide serving table or ML feature.
Who serves an ML model?	MLflow tracks and manages lifecycle; Model Serving or another runtime hosts inference.
What if Gold is incorrect?	Trace lineage; reconcile metrics; inspect Silver / Bronze; rerun controlled backfill.

Final architecture checklist

- Sources and data contracts identified; source keys, grain and change pattern documented
- Incremental ingestion + checkpoint/restart strategy selected
- Bronze raw fidelity and retention/replay policy defined
- Silver data quality, CDC, late-arrival and quarantine rules defined
- Gold metrics, grain, joins and serving contracts documented
- Dashboards and ML flows separated appropriately
- Unity Catalog privileges, lineage, storage security and audit paths established
- Jobs, alerts, tests, CI/CD and recovery / backfill playbook in place

Official Reference Reading

All links below are official Databricks documentation. Product capabilities can vary by cloud, region and release status; check connector support and service availability for your environment.

1. Databricks reference architectures
2. Medallion lakehouse architecture
3. Lakeflow Connect concepts
4. Auto Loader schema inference and evolution
5. Auto Loader FAQ / file mutation and checkpoints
6. Lakeflow pipelines concepts
7. Spark Declarative Pipelines compared with Lakeflow
8. Data quality expectations
9. Unity Catalog overview
10. Unity Catalog data lineage
11. Lakeflow Jobs scheduling and triggers
12. Declarative Automation Bundles
13. Bundle rename FAQ
14. Unity Catalog model lifecycle